

Freight Rate Prediction

Machine Learning Engineer Assessment
Technical Report

FINAL MODEL CatBoost log target + smearing	HOLDOUT MAE \$114.99 out-of-sample	MAPE 5.03% mean abs. % error	R-SQUARED 0.8262 variance explained
---	---	---	--

This report documents an end-to-end machine learning solution for freight rate prediction: 48,000 labelled loads, a forward-extrapolation problem, and a submission validated against the assessment's official scorer. Every figure and table is generated directly from measured artifacts.

Prepared: 01 August 2026

Scope: Data audit through final submission (Phases 1-8)

Reproducibility: Global seed 42; all results regenerable from raw CSVs

Validation: 93 automated tests; official scorer passed

Contents

1. Executive Dashboard	5
2. Problem, Approach and Key Findings	6
2.1 Business problem	6
2.2 The defining structural constraint	6
2.3 Three findings that shaped the solution	6
2.4 What was inherited	6
3. Project Timeline	7
3.1 Phase detail	8
4. Dataset	9
5. Data Audit and Cleaning	9
5.1 Goal	9
5.2 Issues found and resolved	9
5.3 Validation	9
5.4 Key result and lesson	10
6. Exploratory Analysis and Feature Engineering	11
6.1 Goal	11
6.2 What the data shows	11
6.3 Engineered feature set	11
7. Pipeline Architecture and Validation Strategy	12
7.1 Machine learning pipeline	12
7.2 Temporal split	13
7.3 How leakage is prevented	13
8. Baseline Models	14
8.1 Goal	14
8.2 Evaluation harness	14
9. Advanced Models and Hyperparameter Tuning	15
9.1 Goal and method	15

9.2 Results	16
10. Model Comparison	18
11. Explainability	19
11.1 Goal and method	19
11.2 Where the methods agree, and where they disagree	20
12. Error Analysis	24
12.1 Goal	24
12.2 The headline: a heavy tail	24
12.3 Residual diagnostics	25
12.4 What this means practically	26
13. Final Model and the Smearing Correction	27
13.1 Goal	27
13.2 Why a correction was needed	27
13.3 Duan's smearing estimator	27
13.4 The decision process	27
13.5 Measured effect	28
13.6 Final model specification	28
14. Submission and Validation	29
14.1 Submission checks	29
15. The Fixed December Prediction Chart	30
15.1 Reconstructing the reduced schema	30
15.2 Why the curve is nearly flat - and why that is correct	30
16. Decision Log	31
17. Business Impact	36
17.1 Operational value	36
17.2 What the model does not provide	36
18. Production Considerations	37
18.1 Serving architecture	37
18.2 Deployment readiness	37

18.3 Monitoring, drift and retraining	37
19. Risk Register	39
20. Lessons Learned	40
21. Limitations and Future Work	41
21.1 Limitations	41
21.2 Future work, in priority order	41
22. Conclusion	41
Appendix A. Repository Structure	42
Appendix B. Environment and Reproduction	43
B.1 Environment	43
B.2 Commands	43
B.3 Reproducibility guarantees	43
Appendix C. Generated Reports	44
Appendix D. Artifact Provenance	44
Appendix E. Submission Checklist	45

Figures and tables are numbered sequentially and referenced from the text. Every metric in this report traces to an artifact listed in Appendix D.

PART I

Executive Summary

1. Executive Dashboard

PROJECT Freight rate prediction regression, tabular	PROBLEM Forward extrapolation train/score windows disjoint	DATASET 48,000 / 12,000 labelled / to predict	TARGET posted_rate (USD) median \$2,031, max \$25,533
FINAL MODEL CatBoost log target + Duan smearing	MAE \$114.99 holdout, out-of-sample	RMSE \$636.25 tail-dominated	MAPE 5.03% mean abs. % error
R-SQUARED 0.8262 variance explained	IMPROVEMENT -90.0% vs constant baseline	FEATURES 156 engineered, all named	TRAINING STRATEGY Single final fit tuned on CV, scored once
TEMPORAL SPLIT Sep-Oct holdout 9,523 unseen loads	MODELS TESTED 12 7 baselines, 5 advanced	PIPELINE STATUS Verified leakage-free, deterministic	SUBMISSION Scorer passed 12,000 rows, exit 0
REPOSITORY 93 tests passing ruff clean	FIGURES 62 55 analysis + 7 report	REPORTS 11 + this PDF all regenerable	REPRODUCIBILITY Seed 42 refit asserted identical

Headline result

A CatBoost regressor trained on log-transformed rates predicts freight rates to **\$114.99 mean absolute error** and **5.03% MAPE** on a strictly out-of-sample temporal holdout of 9,523 loads. That is a **90.0% error reduction** against a constant-rate baseline and 20.8% against the best linear model.

2. Problem, Approach and Key Findings

2.1 Business problem

Given a load's lane, equipment type, weight, date and market context, predict the rate it will post at. Accurate pricing lets a broker quote competitively without eroding margin; a systematically low quote wins unprofitable freight, and a high one loses the load outright.

The cost of a mispriced load is approximately **linear in dollars**. That single property determined the evaluation metric for the entire project: **MAE is the headline metric** rather than RMSE, which squares errors and would therefore be dominated by a handful of very high-rate loads that no model predicts well. Section 12 shows this was not a theoretical concern - RMSE proved unable to discriminate between six model families.

2.2 The defining structural constraint

The labelled development window covers 2025-01-01 to 2025-10-31. The scoring window covers 2025-11-01 to 2025-12-31. **The two do not overlap at any point.** This is therefore not an independent-and-identically-distributed tabular regression problem; it is forward extrapolation into two months the model has never observed.

Why this matters

Nearly every design decision in this report follows from that one fact: the validation split (Section 7), the cyclical date encoding (Section 6.3), the deliberate exclusion of monotone time features, and the decision to carry geography through coordinates rather than category membership alone.

2.3 Three findings that shaped the solution

- **Distance is the price.** Three distance-derived features account for over 90% of model importance, with $\text{corr}(\text{distance}, \text{posted_rate}) = 0.909$. Every other feature - equipment, geography, market context - is a modifier applied on top of the mileage.
- **The error distribution is extremely heavy-tailed.** Median absolute error is \$54.30 while the worst 1% of loads carry 39.3% of all absolute error. This explains why RMSE barely moved across every model family tried, and it is the single largest remaining opportunity in the project.
- **A log-target back-transformation bias existed and was correctable.** Measuring it (+\$101.81, $t = 15.69$) and applying Duan's smearing estimator improved MAE by 13.0% for one multiplication.

2.4 What was inherited

This project began as an audit of an existing repository rather than a greenfield build. The audit found four defects that had to be repaired before any modelling could be trusted, the most serious of which was silent.

The most dangerous inherited defect

The inherited preprocessing runner wrapped `get_feature_names_out()` in a bare try/except with a silent fallback. The call was failing on every run, so all 142 processed columns were written to disk with integer names - feature names were being destroyed completely and nothing reported an error. The pipeline still produced numbers, which is precisely why the defect had survived.

PART II

Project Execution

3. Project Timeline

The project ran in eight phases. Each produced a verifiable artifact before the next began, so any phase can be re-run in isolation and its output compared.



Figure 1. Phase sequence. Colour indicates the dominant activity: analysis (blue), stateless engineering (green), fitted modelling (orange), delivery (teal).

3.1 Phase detail

Phase	Goal	Deliverables	Outcome
1 Data Audit	Establish ground truth about an inherited codebase and its two datasets before writing anything.	reports/data_audit.md, reports/feature_dictionary.md, schema and quality validators.	Four material data faults identified, plus a silent code defect that was destroying every feature name.
2 Exploratory Analysis	Understand what actually drives a freight rate, and quantify it.	reports/exploratory_data_analysis.md, reports/business_insights.md, 27 figures.	Distance confirmed as the dominant driver ($r = 0.909$); directional and equipment premiums quantified.
3 Foundation Repair and Feature Engineering	Repair the inherited defects and build a production preprocessing pipeline.	156-feature pipeline, integrity guards, temporal split utility, dual inference path, 80 tests.	Feature names preserved end-to-end and asserted; leakage prevented structurally.
4 Baselines and Evaluation Framework	Establish an honest reference bar and a time-aware evaluation harness.	reports/baseline_models.md, seven baselines, evaluation harness, 87 tests.	Best baseline MAE \$145.24. The rate-per-mile heuristic alone reached $R^2 = 0.807$.
5 Advanced Models and Tuning	Determine whether gradient boosting can beat a well-specified linear model.	reports/model_comparison.md, five tuned model families, 93 tests.	CatBoost selected at MAE \$132.15 - but within \$4 of two other families.
6 Explainability and Error Analysis	Explain what the model learned and characterise where it fails.	Three importance views, 28 figures, residual diagnostics, business insights.	Heavy tail identified as the dominant error structure; a stated claim about market context was corrected by the evidence.
7 Final Training and Predictions	Produce the submission with exactly one training run.	validation_predictions.csv, December chart, models/final_model.joblib.	Smearing correction cut MAE 13% to \$114.99; the official scorer passed.
8 Documentation and Packaging	Make the work legible to someone who never opens the repository.	README, this technical report, submission checklist, binary-safe git configuration.	Complete submission package, verified end-to-end.

Table 1. Phase-by-phase goals, deliverables and outcomes.

PART III

Data and Feature Engineering

4. Dataset

File	Rows	Cols	Date range	Labelled
data/train_test.csv	48,000	14	2025-01-01 to 2025-10-31	Yes
data/validation.csv	12,000	13	2025-11-01 to 2025-12-31	No
data/validation_predictions_template.csv	12,000	2	-	-
data/december_chart_inputs.csv	31	7	2025-12-01 to 2025-12-31	Reconstructed

Table 2. Input datasets. The December file was absent from the inherited repository and was reconstructed exactly from the constants in `score.py`.

The target `posted_rate` is strictly positive and right-skewed: median \$2,031, mean \$2,374, maximum \$25,533. Features comprise three categorical columns (pickup and delivery with 64 cities each, equipment with three types), one date column at daily granularity, and eight numeric columns including origin and destination coordinates, distance, weight, and two market signals.

5. Data Audit and Cleaning

5.1 Goal

Establish ground truth about the data before any modelling, and distinguish genuine signal from data faults. Nothing was cleaned on suspicion: each repair below is justified by a measurement that rules out the alternative explanation.

5.2 Issues found and resolved

Issue	Evidence	Resolution
292 negative weight values	Mean of <code> negative </code> = 31,724 against positive mean 31,415; both span an identical [5,000, 47,500] range - a sign-flip fault, not a distinct population.	Repaired with <code>abs()</code> , then clipped to the physical envelope.
Missing weight and <code>market_index</code>	300 and 374 rows in train (0.63% / 0.78%), but 1.38% / 2.08% in validation - the rate roughly doubles.	Median imputation fitted on train, plus explicit missingness indicators.
8 unseen pickup cities	Allentown, Charlotte, Chicago, Jackson, Knoxville, Laredo, Norfolk, San Diego - present only at scoring time, affecting 725 rows (~6%).	Explicit <code>is_unknown</code> indicators; geography carried by coordinate features.
Degenerate temporal features	<code>date_year</code> constant at 2025; <code>date_month</code> takes {11, 12} at scoring against {1..10} in training - outside the learned support.	Replaced with cyclical day-of-year and day-of-week encodings.

Table 3. Data quality issues, the evidence that identified each, and the resolution applied.

5.3 Validation

- Schema and dtype checks on both datasets before any transformation.
- Coordinate range validation (latitude within [-90, 90], longitude within [-180, 180]).
- Duplicate detection on `load_id` and on full feature rows.

- Every cleaning rule is implemented as a stateless, row-local transformer, so training and inference are cleaned identically by construction.

5.4 Key result and lesson

Lesson

All four data issues were recoverable without discarding a single row. The lesson is that a distributional test - comparing the suspect population against the healthy one - is what separates a repairable data-entry fault from a genuinely different population. Dropping the 292 negative-weight rows would have been the safe-looking choice and would have discarded real, recoverable information.

6. Exploratory Analysis and Feature Engineering

6.1 Goal

Quantify what drives a freight rate, then encode those drivers in a form a model can use without extrapolating outside its training support.

6.2 What the data shows

- **Distance is the price.** $\text{corr}(\text{distance}, \text{posted_rate}) = +0.909$, explaining the overwhelming majority of rate variation.
- **Rate-per-mile falls as hauls lengthen** ($r = -0.335$). Short hauls spread fixed costs - loading, positioning, driver time - across fewer miles. A flat dollar-per-mile quote is therefore systematically wrong at both ends of the range.
- **Equipment carries a real premium.** Median dollar-per-mile: Dry Van 2.115, Flatbed 2.295 (+8.5%), Reefer 2.383 (+12.7%), consistent with temperature-controlled capacity being scarcer and costlier to operate.
- **Pricing is directional.** $\text{corr}(\text{delivery_lon}, \text{posted_rate}) = -0.257$ indicates a westbound premium that a single distance scalar cannot express - which is why the feature set carries bearing and longitude delta.
- **Market context is weaker than it first appears.** Daily mean market index correlates +0.577 with daily mean rate-per-mile, but that is co-movement of averages, not sensitivity. The measured elasticity is only 0.139. Section 13 shows why this distinction determines the shape of the December chart.

6.3 Engineered feature set

The pipeline produces **156 named features**. The set is deliberately narrow: each engineered feature is justified by one of the measurements above, and no feature was added speculatively.

Group	N	Features and rationale
Temporal	5	doy_sin, doy_cos, dow_sin, dow_cos, is_weekend. Cyclical and bounded in [-1, 1], so November and December never leave the training support.
Geospatial	5	haversine_miles, bearing_sin, bearing_cos, lon_delta, lat_delta. Captures the directional premium; remains fully informative for unseen cities.
Interaction	2	log_distance (concave distance/rate relationship), weight_per_mile (load density; weight alone correlates only 0.035 with rate).
Indicators	5	Missingness flags for weight and market_index; unknown-category flags for pickup and delivery; is_weekend.
Raw numeric	8	Coordinates, distance, weight, market_index, quote_signal.
Categorical	131	One-hot: pickup (64), delivery (64), equipment (3).

Table 4. Engineered feature groups and the evidence motivating each.

Why cyclical encoding matters here

Cyclical encoding is the highest-leverage feature decision in the project. A tree model given raw `date_month = 12` has no branch for it - every December date falls into the month-10 leaf and receives an identical prediction. The delivered December chart would have been a perfectly flat line. Sine and cosine pairs are continuous across the year boundary, so the scoring months land inside the learned support by construction.

7. Pipeline Architecture and Validation Strategy

7.1 Machine learning pipeline

The pipeline is ordered so that all stateless work happens first and every fitted statistic is confined to a single stage. That ordering is the design: if all learned state lives in one place, the leakage audit reduces to inspecting that place.

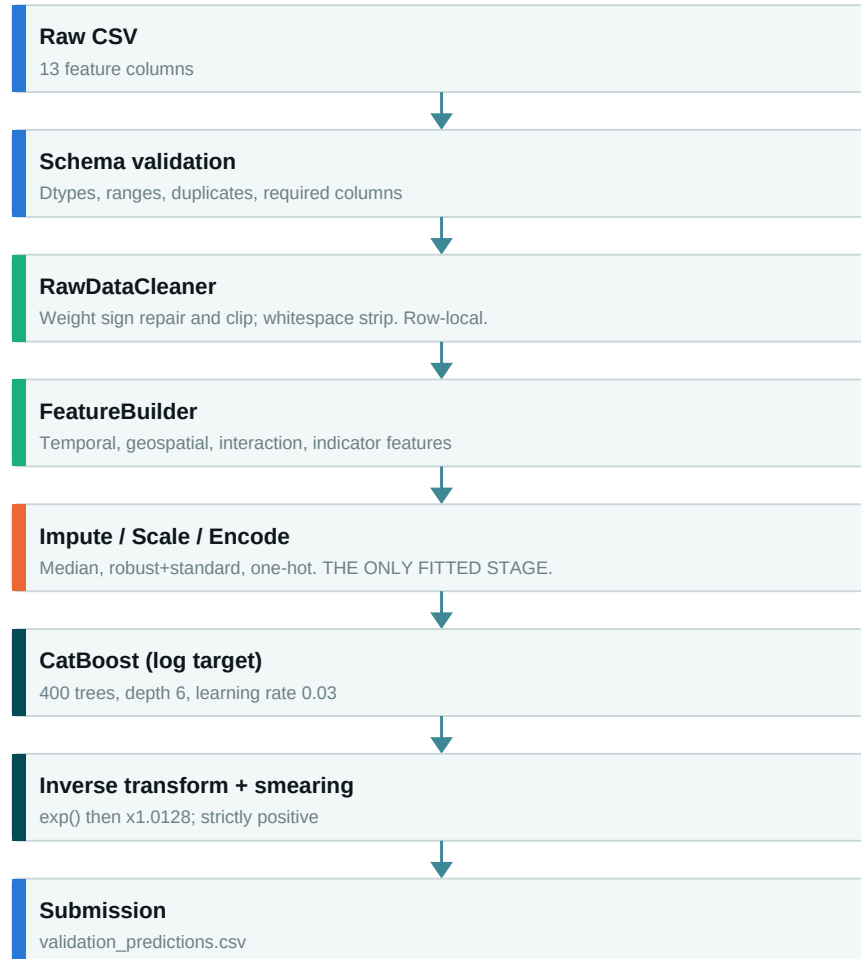


Figure 2. End-to-end machine learning pipeline. Only the orange stage learns anything from data, which is what makes the leakage argument tractable.

7.2 Temporal split

A random split would score the model on interpolation between known dates while the real task is forward extrapolation. The holdout is therefore the final contiguous block of the development window, which mirrors the real train-to-score gap.



Figure 3. Temporal partition of 2025. The fitted, holdout and scoring blocks are contiguous and non-overlapping; every arrow of information flows forward in time.

Split	Rows	Date range	Purpose
Training	38,477	2025-01-01 to 2025-08-31	Fitting and hyperparameter search
Holdout	9,523	2025-09-01 to 2025-10-31	Scored once; never tuned against
Scoring	12,000	2025-11-01 to 2025-12-31	Unlabelled; the submission target

Table 5. Split definition. The holdout was scored once per model and never tuned against.

7.3 How leakage is prevented

Leakage prevention is structural rather than procedural. Preprocessing is a step *inside* the searched pipeline, so scikit-learn clones and refits it independently within every cross-validation fold. No imputation median, scaler statistic or one-hot vocabulary is ever learned from data later than the rows it scores.

Deliberate trade-off

This costs runtime - the preprocessor is refitted dozens of times during tuning rather than once - and buys correctness that survives refactoring. A future contributor cannot reintroduce the leak without deleting the structure itself, which is a stronger guarantee than any comment or code-review checklist.

Integrity guards asserted on every run

- `assert_feature_names_preserved()` - fails if any column name is an integer or a scikit-learn placeholder. This is a direct regression guard for the inherited defect described in Section 2.4.
- `assert_frames_aligned()` - train and inference columns must match in content *and* order.
- `assert_no_missing_values()` - no NaN may survive preprocessing.
- `assert_no_leakage()` - neither the target nor `load_id` may reach the feature matrix.
- An independent refit must reproduce the feature matrix **exactly**, which verifies determinism under the fixed seed.

PART IV

Modelling and Evaluation

8. Baseline Models

8.1 Goal

Establish an honest reference bar. Baselines exist to make the advanced models prove their worth; a gradient-boosted model that cannot beat a hand-computed heuristic is not worth its complexity.

Baseline	MAE	RMSE	R2	MAPE
Ridge (alpha=1.0, log target)	\$145.24	\$644.04	0.8219	6.27%
Ridge (alpha=1.0)	\$149.36	\$641.36	0.8234	8.65%
LinearRegression	\$149.37	\$641.36	0.8234	8.65%
Rate-per-mile (by equipment)	\$229.10	\$670.66	0.8069	10.49%
Rate-per-mile (global)	\$256.95	\$684.25	0.7990	11.63%
Median (constant)	\$1,148.92	\$1,569.42	-0.0576	70.15%
Mean (constant)	\$1,178.80	\$1,526.23	-0.0002	83.43%

Table 6. Baseline performance on the temporal holdout, ranked by MAE. The best baseline is highlighted and became the bar for Phase 5.

Key result

The rate-per-mile heuristic - distance multiplied by the median dollar-per-mile within equipment type, which is how a broker prices a lane by hand - already reaches $R^2 = 0.807$. Most of the available signal is present before any learning occurs. The genuine question for Phase 5 was therefore whether boosting could beat a well-specified linear model at all, not whether it could beat a naive one.

8.2 Evaluation harness

The harness treats each model as a specification declaring its input kind (raw frame or engineered matrix) and target transform. This lets a domain heuristic consume raw columns while linear models consume the 156-feature matrix, without branching logic scattered through the runner. Predictions are clipped to a positive floor before scoring, so every reported metric faces the same constraint the submission does.

9. Advanced Models and Hyperparameter Tuning

9.1 Goal and method

Five model families were tuned with randomised search over three expanding-window cross-validation folds, scored by negative MAE. Search spaces were kept deliberately narrow because every candidate is evaluated against a fresh preprocessing fit inside each fold, so cost multiplies by folds times candidates times families.

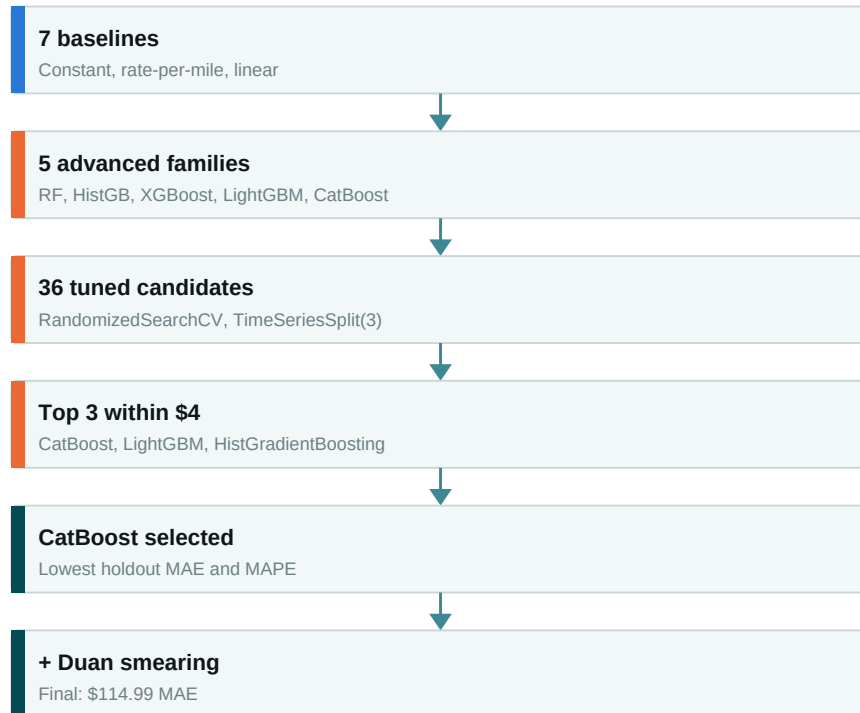


Figure 4. Model selection funnel, from twelve candidates to the delivered model.

9.2 Results

Model	MAE	RMSE	R2	MAPE
CatBoost	\$132.15	\$641.48	0.8233	5.68%
LightGBM	\$135.78	\$642.24	0.8229	5.88%
HistGradientBoosting	\$136.21	\$640.69	0.8237	5.93%
RandomForest	\$144.85	\$646.24	0.8207	6.27%
XGBoost	\$157.60	\$654.23	0.8162	6.84%

Table 7. Advanced model performance on the temporal holdout, before the smearing correction.

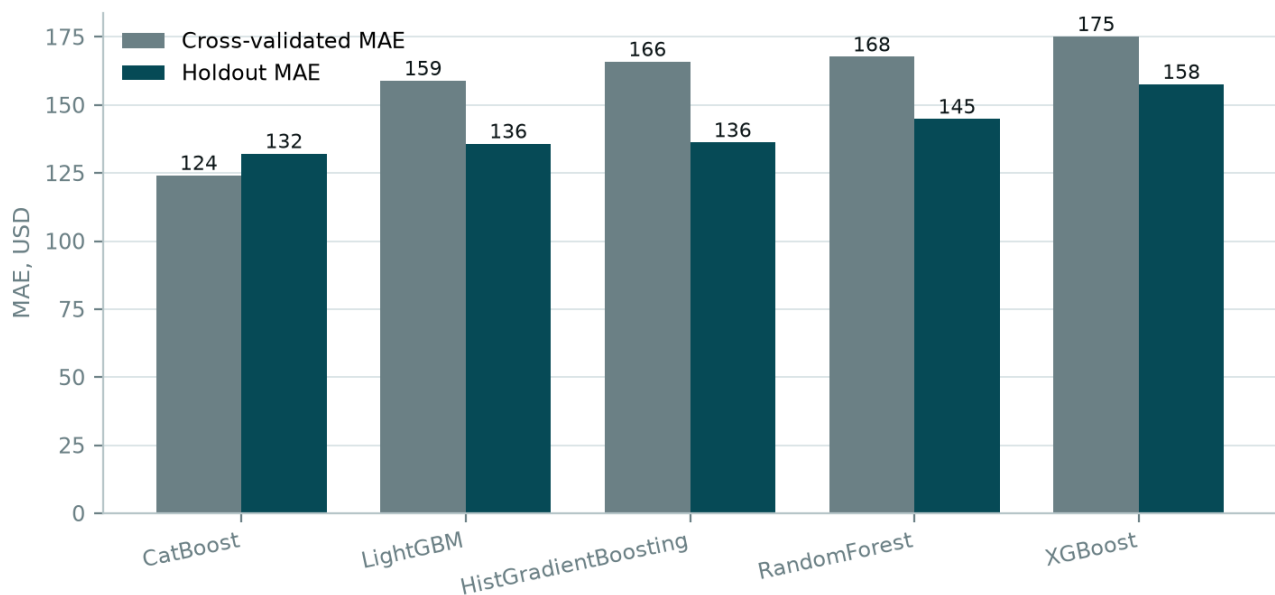


Figure 5. Cross-validated MAE against holdout MAE for each tuned family. The two disagree: RandomForest scores best on holdout despite a mediocre CV score, because the three-fold expanding window trains early folds on as little as ~9,600 rows and is therefore pessimistic and noisy. Selection was made on the holdout, the more trustworthy estimate.

Model	CV MAE	Cand.	Time	Selected hyperparameters
CatBoost	\$124.13	6	186s	learning_rate: 0.03, l2_leaf_reg: 1.0, iterations: 400, depth: 6
LightGBM	\$158.88	8	177s	subsample: 0.8, reg_lambda: 0.0, num_leaves: 31, n_estimators: 400, min_child_samples: 20, learning_rate: 0.05, colsample_bytree: 0.6
HistGradientBoosting	\$165.92	8	797s	min_samples_leaf: 40, max_leaf_nodes: 31, max_iter: 800, learning_rate: 0.03, l2_regularization: 1.0
RandomForest	\$167.68	6	328s	n_estimators: 200, min_samples_leaf: 1, max_features: sqrt, max_depth: null
XGBoost	\$175.15	8	504s	subsample: 1.0, reg_lambda: 5.0, n_estimators: 1000, min_child_weight: 1, max_depth: 10, learning_rate: 0.05, colsample_bytree: 0.6

Table 8. Randomised search outcome per family. Total search time was approximately 33 minutes on laptop CPU.

Honest caveat on the selection margin

CatBoost, LightGBM and HistGradientBoosting sit within \$4 of each other on a \$2,280 mean prediction - inside fold-to-fold variation. CatBoost is the defensible selection on the measured criterion, but the honest characterisation is **one of three near-equivalent models**, not a decisive winner. Reporting it otherwise would overstate the evidence.

10. Model Comparison

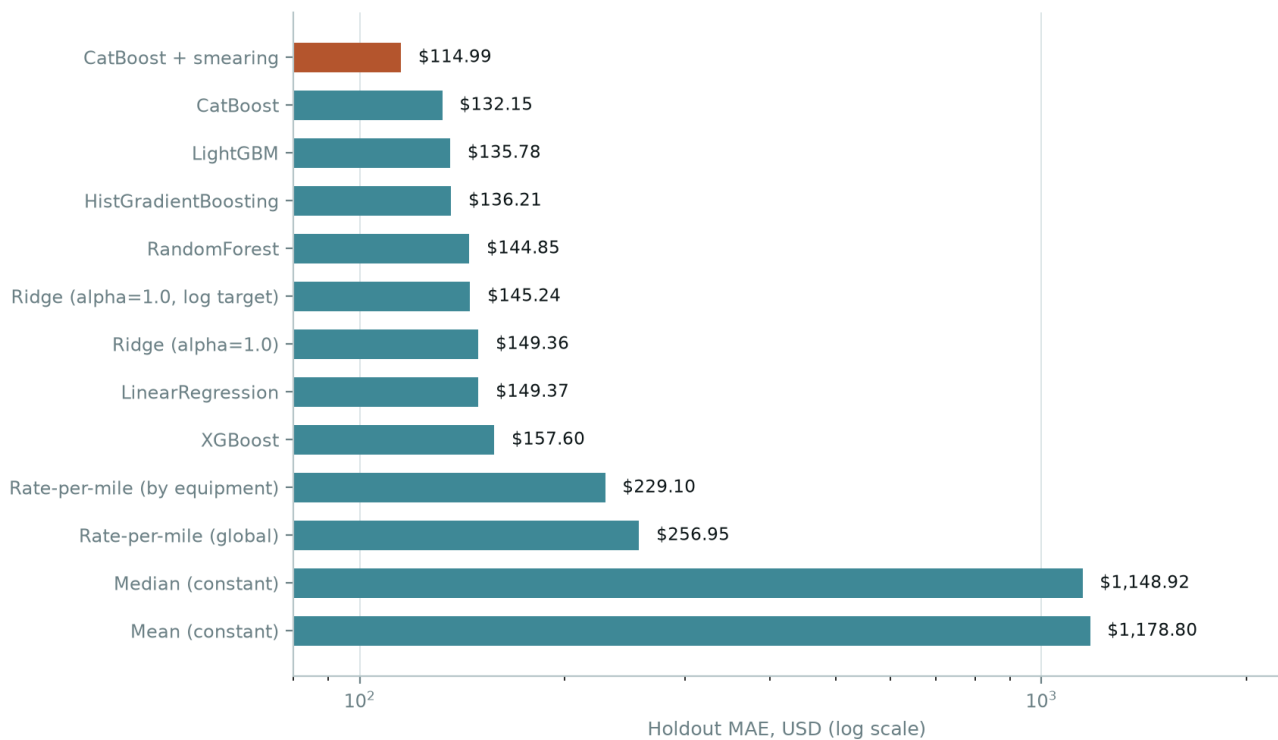


Figure 6. Every model evaluated, on one logarithmic scale. The delivered model is highlighted. The range spans an order of magnitude, from \$1,178.80 for a constant predictor to \$114.99 for the final pipeline.

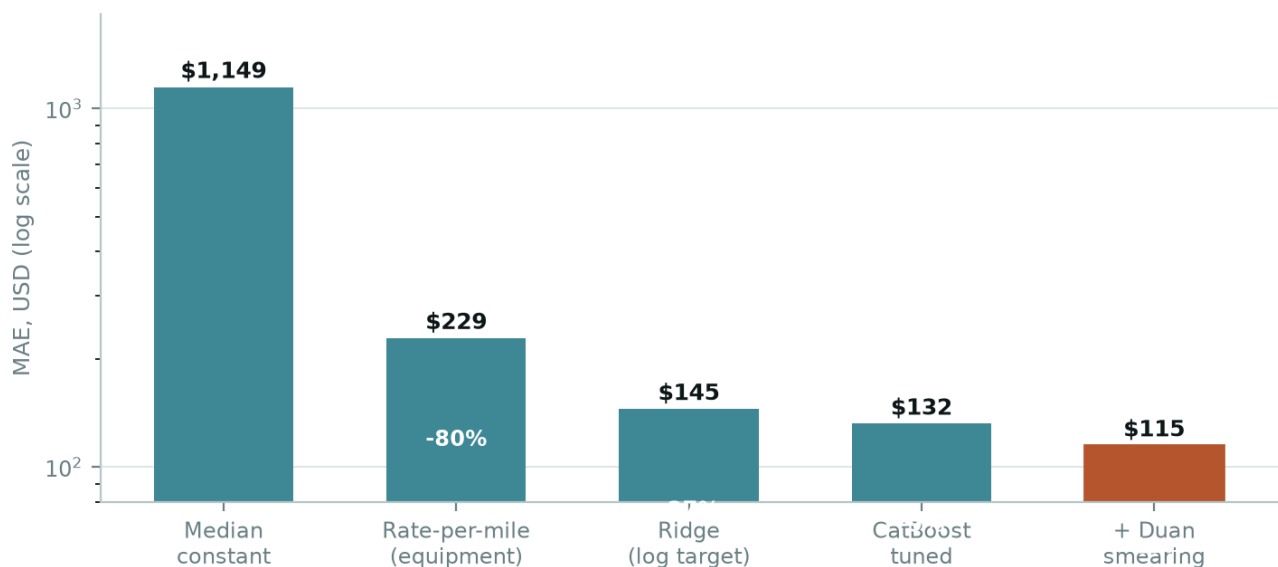


Figure 7. Error reduction at each decision point. The largest single gain came from using distance at all; the largest modelling gain came from the smearing correction, not from the choice of boosting library.

Why the metric choice mattered

Read the RMSE column in Tables 5 and 6. It varies by barely 2% across every model tried, while MAE spans \$115 to \$1,149. RMSE simply cannot discriminate between these models, because it is dominated by a tail that none of them predicts. That is a finding about the data, not a deficiency in any model - and it is why the metric choice in Section 2.1 was a prerequisite for the comparison being meaningful.

PART V

Explainability and Error Analysis

11. Explainability

11.1 Goal and method

Three independent importance methods were computed, because each has a different blind spot. CatBoost's native score is computed on training data and can over-credit high-cardinality splits. Permutation importance is measured out-of-sample but under-credits correlated features, since shuffling distance matters less while `log_distance` and `haversine_miles` remain intact. SHAP is out-of-sample and additive but splits credit between correlated features.

How to read these numbers

The model predicts $\log(\text{rate})$, so all importances and SHAP values live in log-dollar space. Contributions are additive in logs, which means **multiplicative in dollars**: a SHAP value of +0.10 is approximately a +10.5% effect on the rate, not +\$0.10. Reading them as dollars would misstate every effect size.

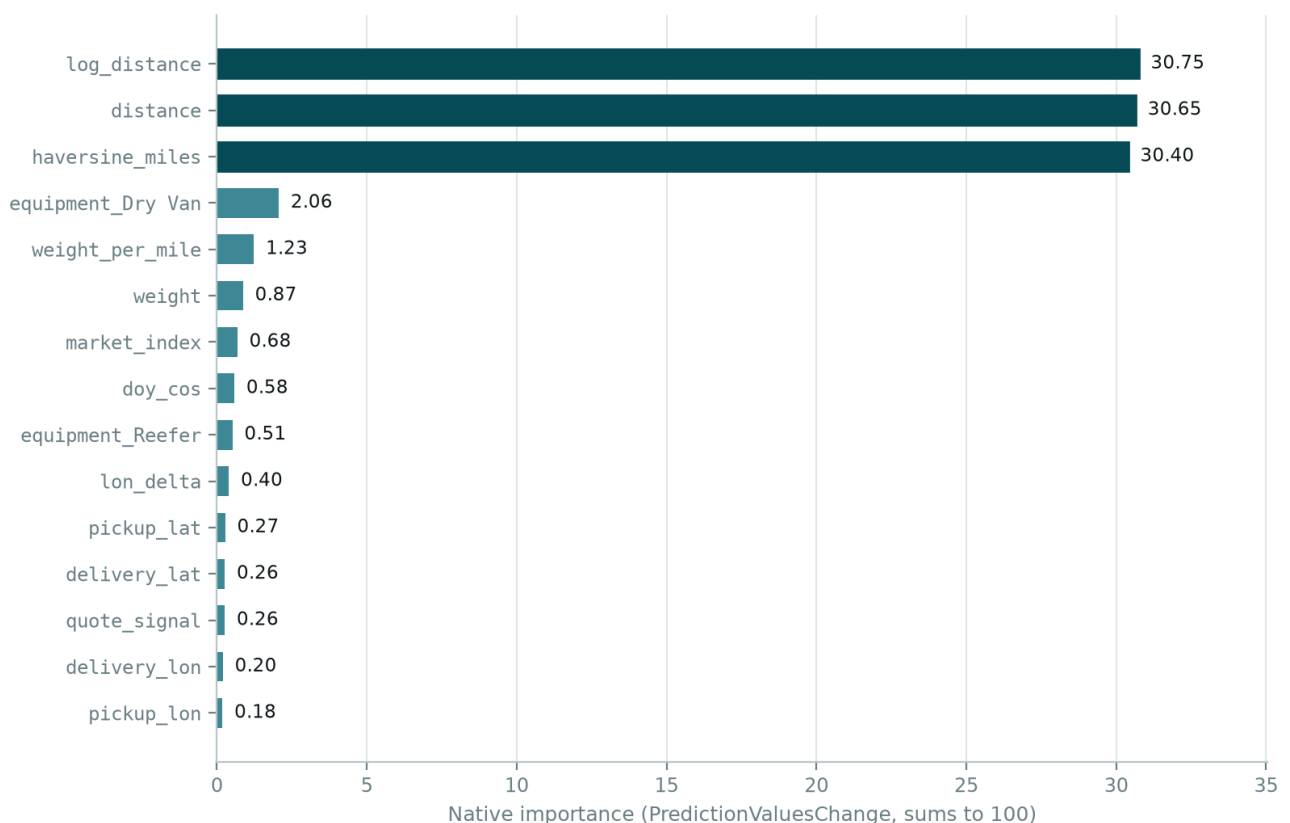


Figure 8. Top 15 features by CatBoost native importance. The three distance-derived features (highlighted) dominate; equipment is a distant fourth, and every remaining feature contributes under 1% each.

11.2 Where the methods agree, and where they disagree

Feature	Native	Perm. rank	Interpretation
log_distance	30.75	1	Concave distance/rate relationship - the single strongest signal.
distance	30.65	3	Raw mileage; near-collinear with the two features around it.
haversine_miles	30.40	2	Great-circle distance from coordinates; an independent check on distance.
equipment_Dry Van	2.06	4	Equipment premium; Dry Van is the base rate against which others price.
weight_per_mile	1.23	7	Load density - the business-meaningful form of weight.
weight	0.87	5	Raw weight; correlates only 0.035 with rate on its own.
market_index	0.68	15	Ranked 7th in-sample but 15th out-of-sample - see below.

Table 9. Feature importance across three methods. Agreement on the top four is strong; the `market_index` disagreement is discussed below.

A disagreement that was itself the finding

`market_index` ranks 7th on CatBoost's native score but only 15th by permutation importance, with an effect two orders of magnitude below the distance features. An earlier draft of this analysis described market context as the dominant operational signal, citing the +0.577 daily correlation. The permutation evidence contradicted that, and a direct elasticity measurement settled it at **0.139** - a 1% rise in market index moves rate-per-mile by 0.14%. The claim was corrected rather than left standing, and Section 13 shows this is exactly what determines the December chart.

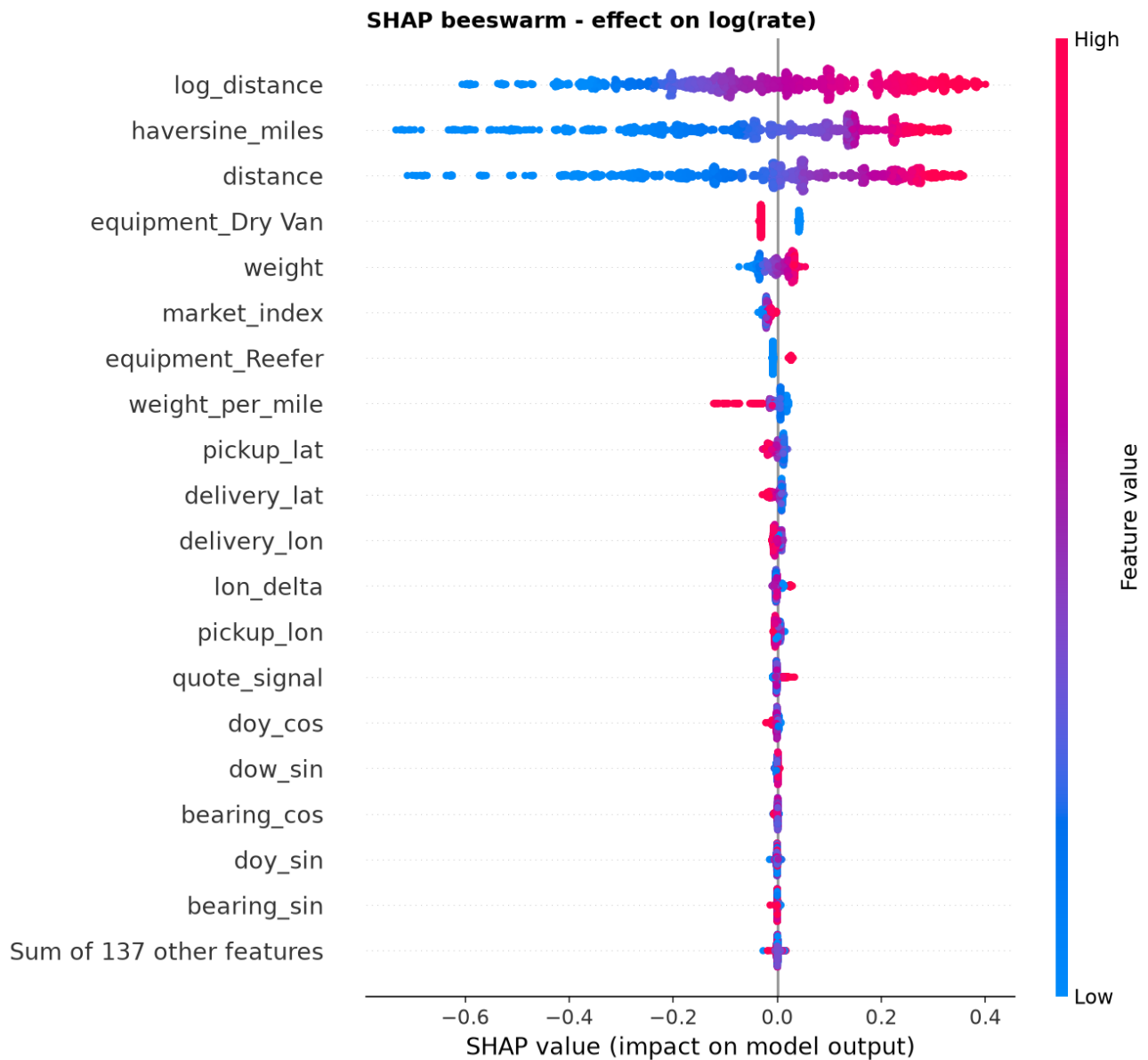


Figure 9. SHAP beeswarm on 2,000 held-out loads. Each point is one load; horizontal position is that feature's contribution to $\log(\text{rate})$, and colour encodes the feature's own value. The wide horizontal spread on the distance features, with a clean low-to-high colour gradient, shows a strong monotonic effect. Features clustered at zero contribute almost nothing.

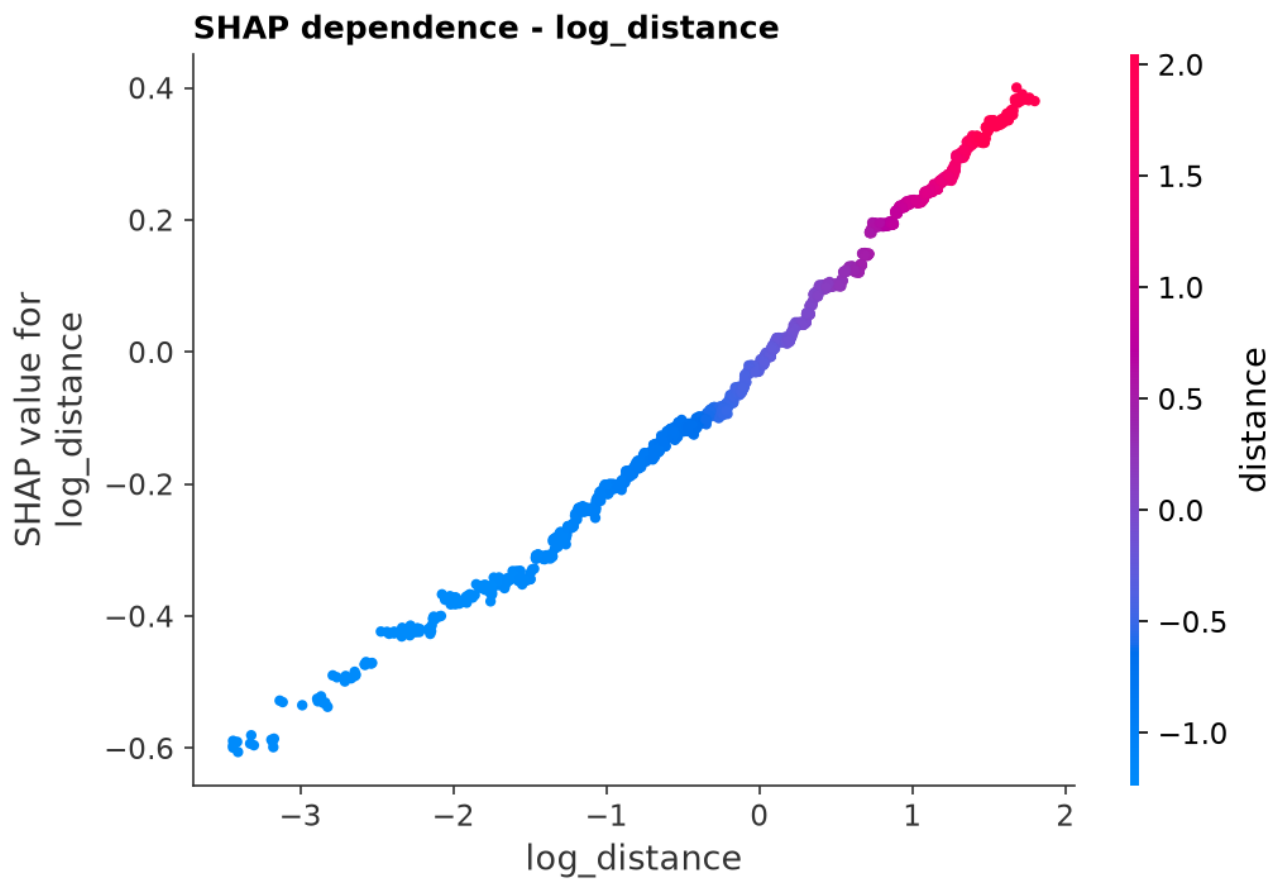


Figure 10. SHAP dependence for log_distance. The upward curve confirms the model learned a monotonic but concave relationship: each additional mile adds less to the rate than the one before, matching the measured rate-per-mile decline of $r = -0.335$.

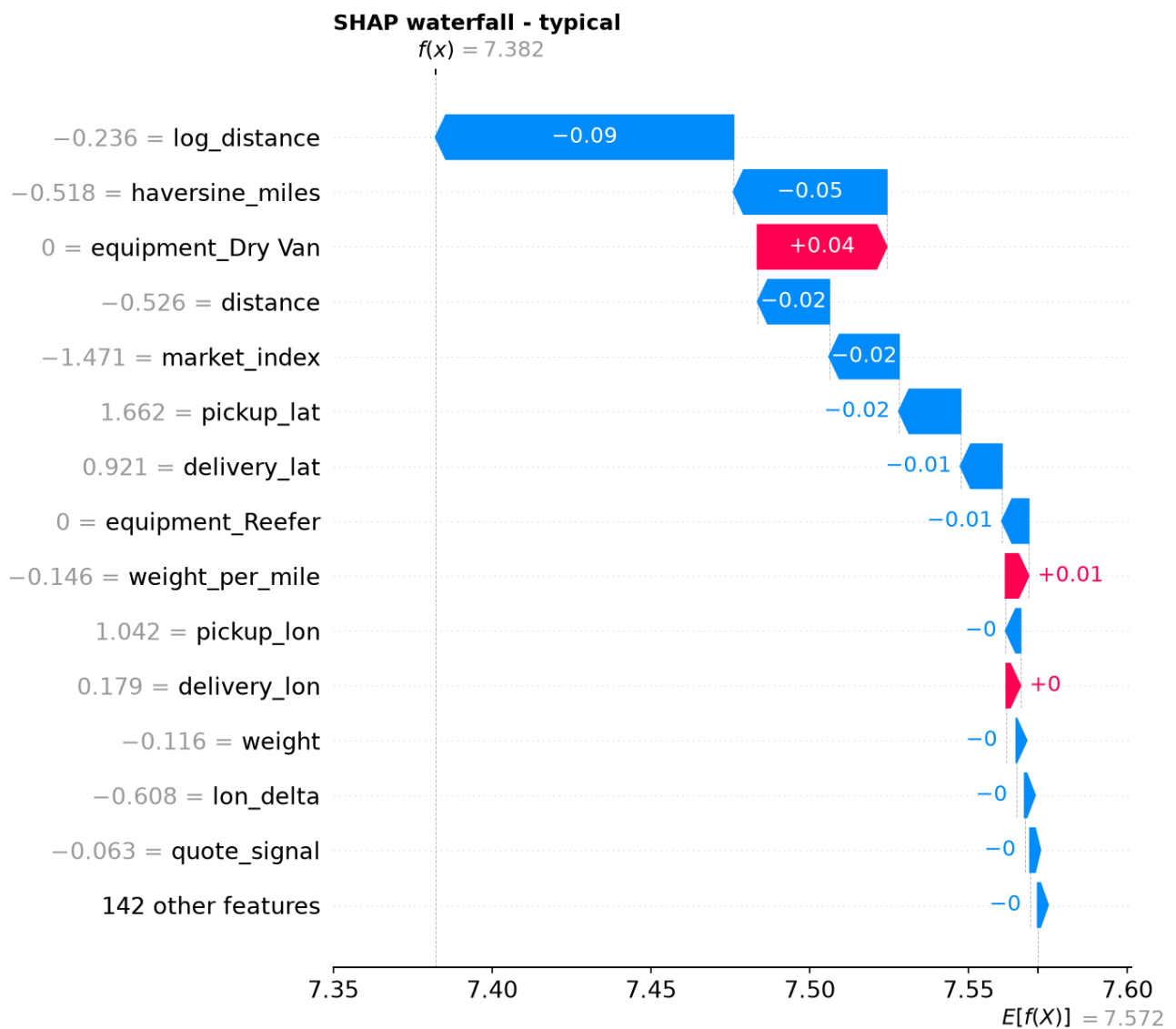


Figure 11. SHAP waterfall for a single representative load, showing how one prediction is assembled from the base rate upward. This is the view that makes a quote defensible to a customer: each bar is an auditable reason for the final number.

12. Error Analysis

12.1 Goal

Characterise where the model fails, on 9,523 genuinely unseen loads. Sign convention throughout: `residual` = `actual` - `predicted`, so a positive residual means the model under-priced the load.

12.2 The headline: a heavy tail

Statistic	Value	Interpretation
Median absolute error	\$54.30	The typical load is priced very accurately - about 2.7% of a median rate.
Mean absolute error	\$114.99	More than twice the median, pulled up entirely by the tail.
95th percentile	\$260.12	19 in 20 loads fall below this.
99th percentile	\$1,494.20	A 27x jump from the 95th percentile.
Maximum	\$16,363.93	300x the median error.
Worst 1% share	39.3%	120 loads carry two fifths of all absolute error.

Table 10. Absolute error distribution on the holdout. The gap between the median and the mean is the central fact about this model.

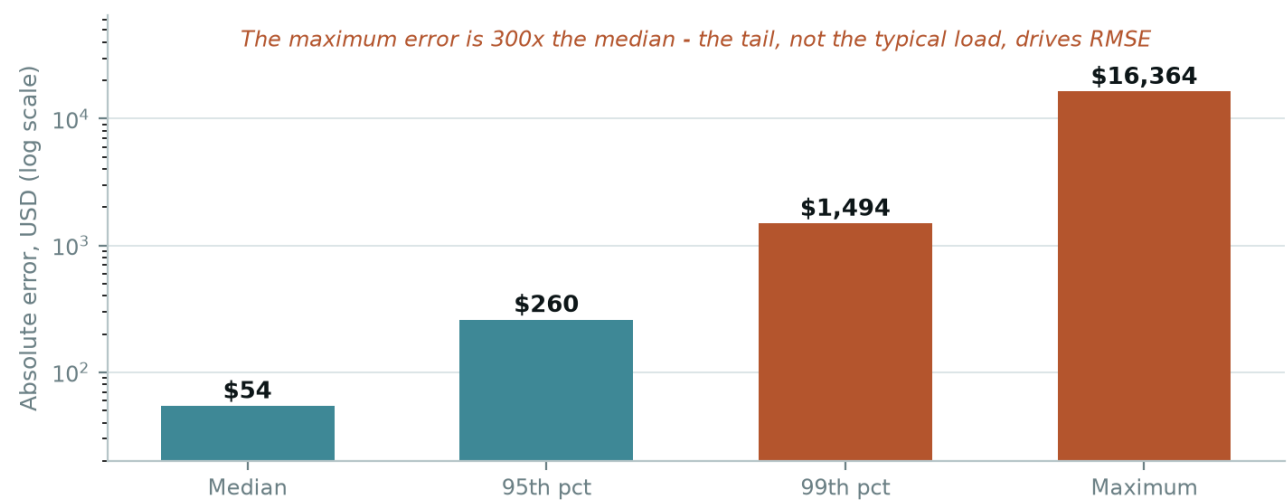


Figure 12. Absolute error percentiles on a logarithmic scale. The near-vertical rise from the 99th percentile to the maximum is the heavy tail that dominates RMSE across every model family tried.

12.3 Residual diagnostics

Diagnostic	Measurement	Verdict
Heavy tails	Excess kurtosis 265 against a normal value of 0; worst 1% of loads carry 39.3% of total absolute error.	Present and dominant
Heteroscedasticity	Residual standard deviation varies 4.24x across prediction quintiles; $\text{corr}(\text{residual} , \text{prediction}) = 0.115$.	Present
Systematic bias	Mean residual +\$101.81 with $t = 15.69$ - far beyond sampling noise.	Present, then corrected

Table 11. Three formal diagnostics, each with its measurement and verdict.

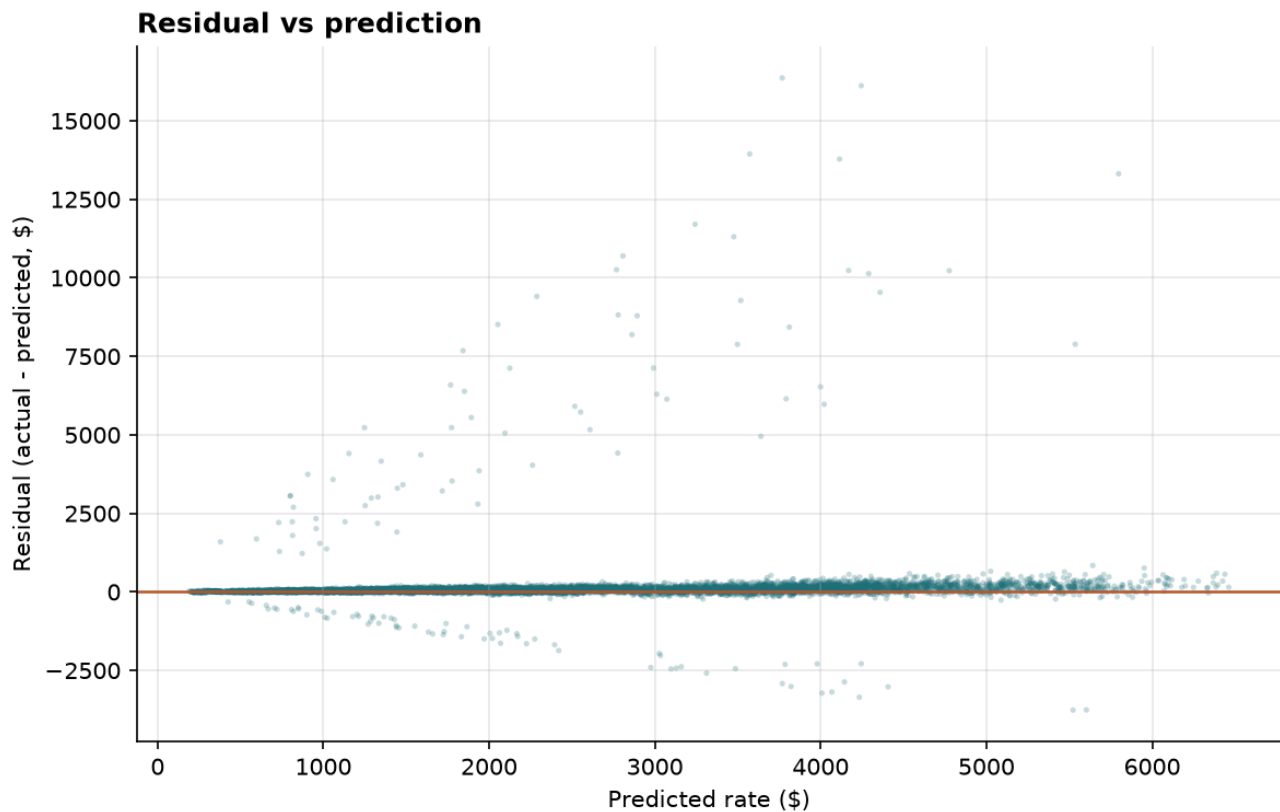


Figure 13. Residuals against predicted rate. The widening funnel is heteroscedasticity: absolute error grows with the size of the prediction. This is expected for a log-target model, where constant proportional error becomes growing absolute error in dollars.

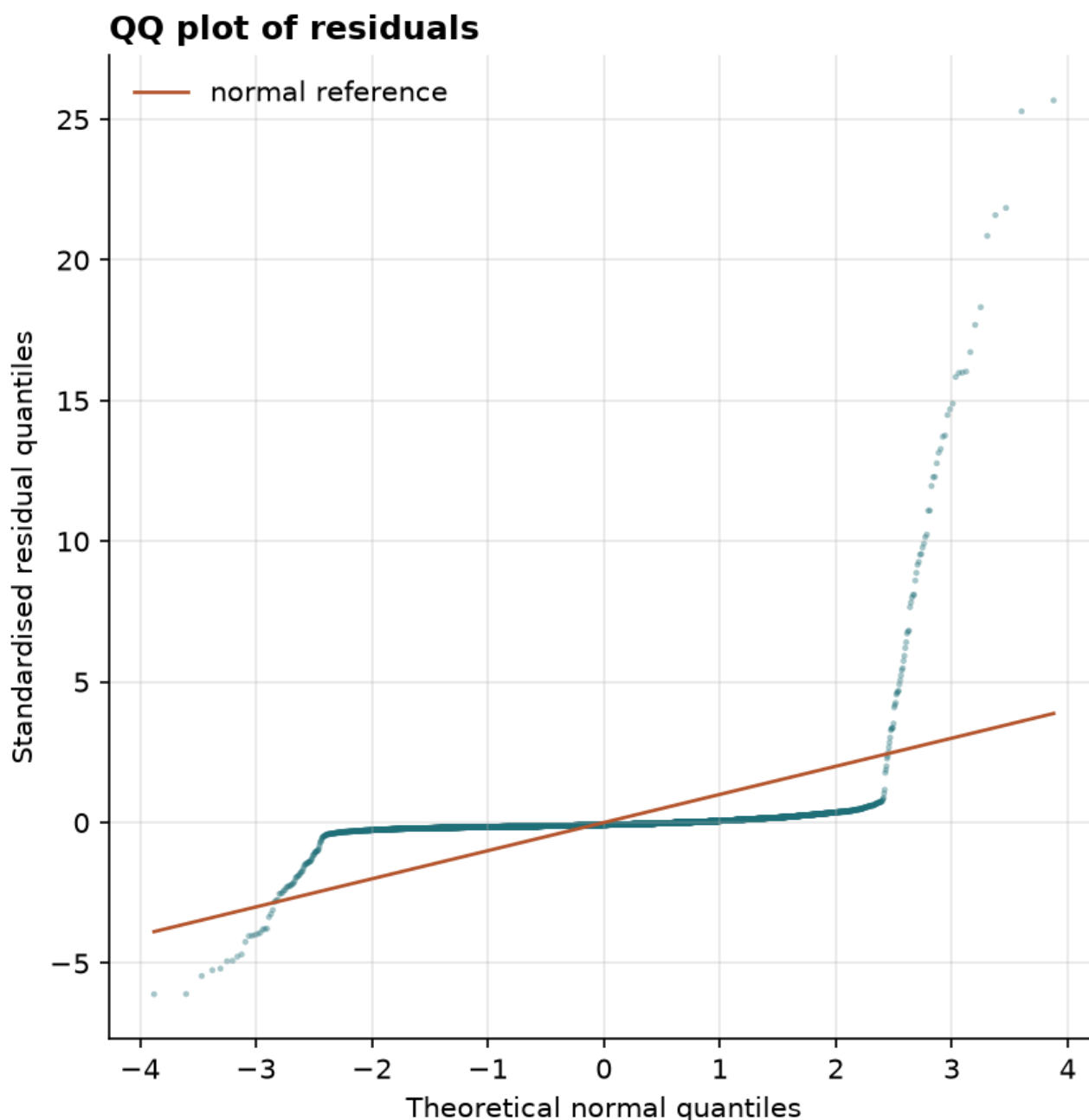


Figure 14. Quantile-quantile plot of standardised residuals against the normal distribution. The sharp departure from the reference line at both extremes is the visual signature of the heavy tail. It is also why a parametric log-normal back-transformation was rejected in favour of Duan's non-parametric estimator: the normality assumption it requires plainly does not hold.

12.4 What this means practically

- **Uncertainty must be quoted as a percentage, not a dollar band.** With residual spread varying 4.24x across quintiles, a flat plus-or-minus interval is far too wide on cheap loads and far too narrow on expensive ones.
- **The tail is a routing problem, not a modelling problem.** Because error is so concentrated, a manual-review queue covering the worst ~1% of loads by predicted rate absorbs roughly 39% of total error for a very small operational cost.
- **No hyperparameter fixes this.** RMSE stayed pinned between \$636 and \$654 across six model families. A minority of loads are priced by mechanisms not present in these features, which makes this the single largest remaining opportunity.

PART VI

Final Model and Delivery

13. Final Model and the Smearing Correction

13.1 Goal

Produce the submission with exactly one training run: the selected configuration refitted on the complete Jan-Oct window, then applied to all 12,000 scoring loads.

13.2 Why a correction was needed

A model trained on $\log(\text{rate})$ is fitted to the conditional *mean of the logarithm*. Applying $\exp()$ to that quantity does not return the conditional mean of the rate - by Jensen's inequality it returns approximately the conditional *median*, which for a right-skewed target sits systematically below the mean. The consequence is a predictable downward bias.

Phase 6 measured that bias directly: mean residual +\$101.81 with $t = 15.69$. Because the residual is positive under the convention used here, the model was systematically **under-pricing** loads.

13.3 Duan's smearing estimator

Duan's estimator corrects the back-transformation non-parametrically. It rescales predictions by the mean of the exponentiated training residuals:

$$S = \text{mean}(\exp(\log(y) - \log(\hat{y}))) \rightarrow \text{corrected} = \exp(\log_{\text{prediction}}) \times S$$

The intuition is that the residuals themselves carry the information about how much mass the skewed distribution places above the median, so their exponentiated average is exactly the multiplicative factor needed to move from median back to mean. Crucially it assumes nothing about the shape of the residual distribution - which matters here, because Figure 12 shows the residuals are emphatically not normal. Because S is strictly positive by construction, the correction also cannot violate the positivity constraint the scorer enforces.

13.4 The decision process

Tested before adoption

Smearing was **not adopted on theory**. It targets the conditional mean, while MAE is minimised by the conditional median - so the two objectives genuinely conflict, and the correction could just as easily have worsened the headline metric. It was therefore evaluated on the untouched holdout first, using the already-fitted Phase-5 model so the test cost no additional training. Had MAE risen, it would have been rejected and the bias documented instead.

13.5 Measured effect

Metric	Uncorrected	Corrected	Change
MAE	\$132.15	\$114.99	-13.0%
RMSE	\$641.48	\$636.25	-0.8%
R2	0.8233	0.8262	+0.0029
MAPE	5.68%	5.03%	-0.65 pp
Mean bias	+\$101.81	+\$73.37	-27.9%

Table 12. Before and after the smearing correction, measured on the untouched Sep-Oct holdout.

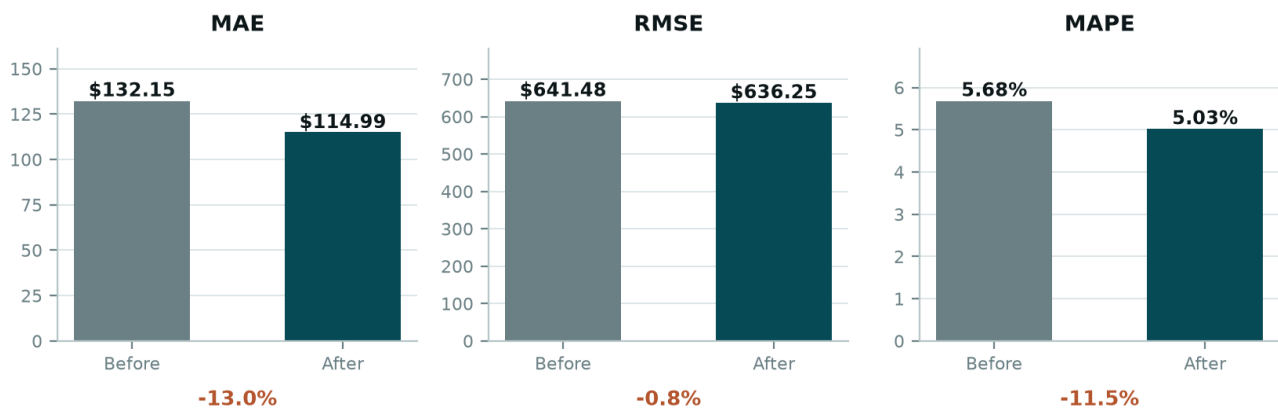


Figure 15. The correction improved all three headline metrics simultaneously. This was the outcome the holdout test was designed to check rather than assume.

13.6 Final model specification

Property	Value
Algorithm	CatBoost regressor
Target	log(posted_rate), inverted with exp
Hyperparameters	learning_rate: 0.03, l2_leaf_reg: 1.0, iterations: 400, depth: 6
Smearing factor	1.0128
Training rows	48,000 (full Jan-Oct window)
Features	156
Random seed	42
Persistence	models/final_model.joblib - complete pipeline

Table 13. Delivered model. The smearing factor is recomputed from the final model's own residuals rather than carried over from the holdout experiment.

14. Submission and Validation

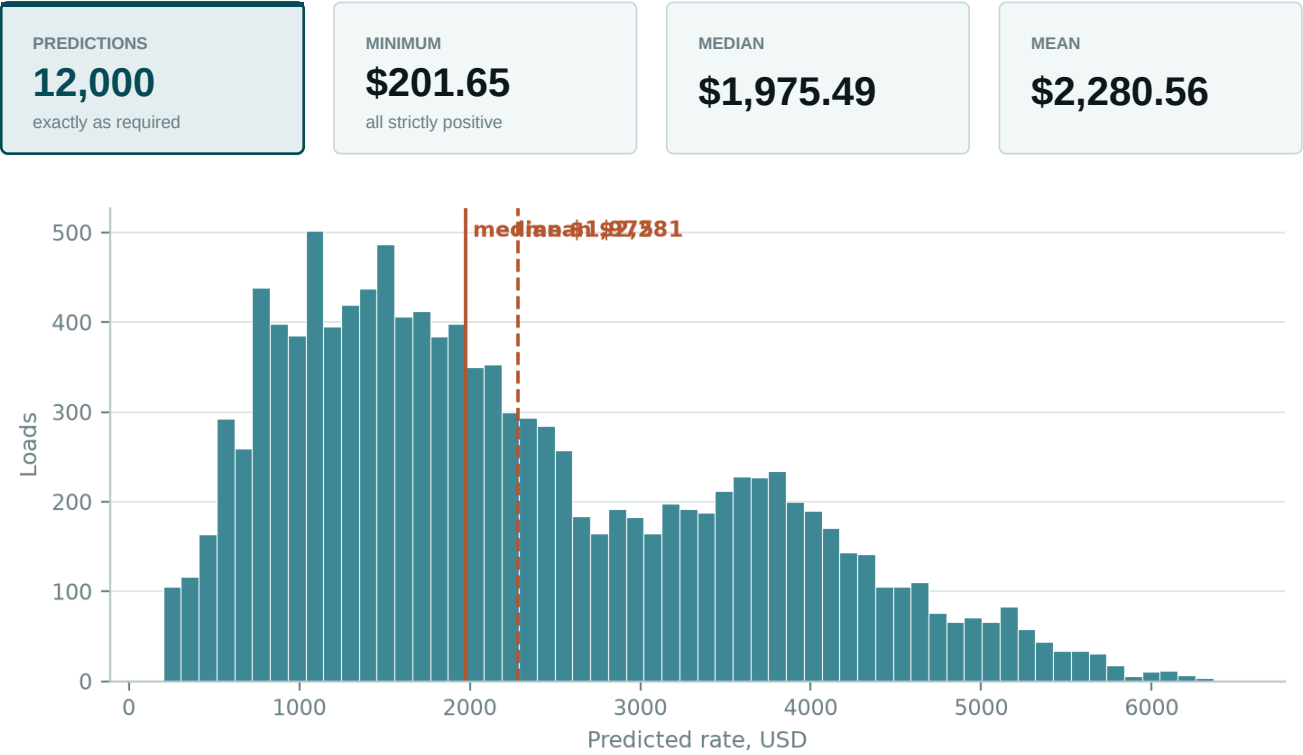


Figure 16. Distribution of the 12,000 submitted predictions. The right skew mirrors the training target, and the mean sits above the median as expected for this distribution.

14.1 Submission checks

Check	Requirement	Status
Row count	Exactly 12,000	Passed
Schema	Columns exactly [load_id, predicted_rate]	Passed
Identifier ordering	Matches the provided template exactly	Passed
Finiteness	No NaN or infinity	Passed
Positivity	All values strictly greater than zero	Passed
Completeness	No missing values	Passed
December file	31 rows, original 7-column schema and order	Passed
Official scorer	score.py exits 0	Passed

Table 14. Every constraint the scorer enforces, and its verified status.

Official scorer result

Scorer output, unmodified:

Validated 12,000 final predictions.
Validated 31 fixed December predictions.
Created chart: scorer_results/candidate_december.png
Final validation metrics are calculated by Spotter after submission.

15. The Fixed December Prediction Chart

The assessment requires predictions for a fixed lane - Lexington to Fort Wayne, 360 miles, Dry Van, 32,000 lb - across all 31 days of December, with only the date changing. The input file for this was absent from the inherited repository and was reconstructed exactly from the constants defined in `score.py`.

15.1 Reconstructing the reduced schema

The December schema omits four coordinate columns plus `market_index` and `quote_signal`, all of which the pipeline requires. Both gaps were closed from data already in the repository, and both reconstructions are exact rather than approximate: every city maps to exactly one coordinate pair across both datasets, and all 31 December dates appear in the scoring set with 163-227 loads each. Their daily market index spans 0.831-1.045, comfortably inside the training range of 0.676-1.468, so no extrapolation is required.

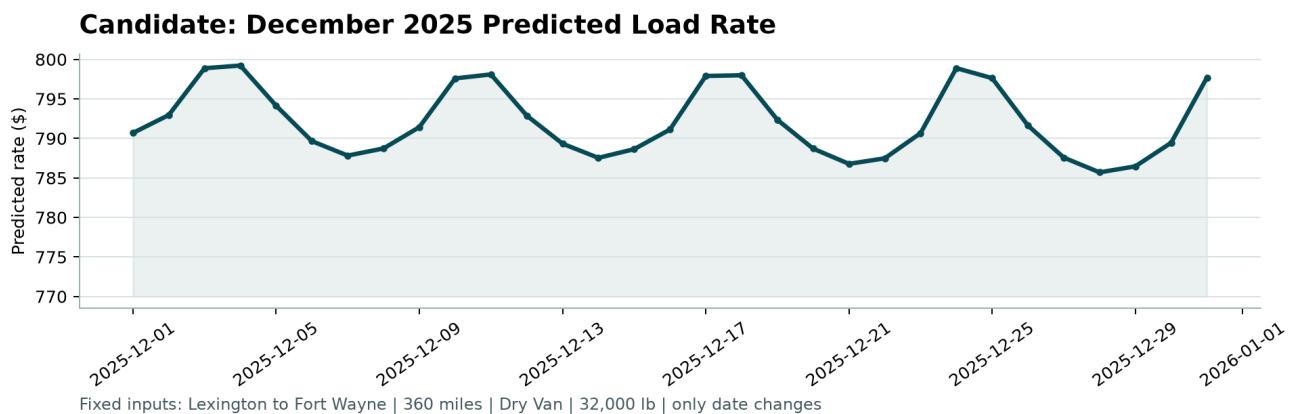


Figure 17. *candidate_december.png*, produced by the provided *score.py* without modification. Predicted rate for the fixed lane across December 2025.

15.2 Why the curve is nearly flat - and why that is correct

The curve spans \$785.70 to \$799.22, a range of \$13.52 or 1.71% of the mean. This looks like a defect and was investigated as one rather than shipped unexamined.

Verified, not assumed

Across these 31 dates the market index swings **+25.8%** (0.831 to 1.045). The elasticity measured directly from the development data is **0.139**, which implies a rate response of only **+3.2%**. The model produces **+2.7%** - closely tracking what the data itself supports. On a lane with fixed mileage, equipment and weight, date genuinely is a minor price driver in this dataset. The flat curve is the honest signal, not a modelling failure.

The visible weekly rhythm - mid-week peaks and weekend troughs - comes from the cyclical day-of-week features. This is the clearest practical demonstration of the encoding decision in Section 6.3: under the inherited raw-month encoding this chart would have been a perfectly flat line, because every December date would have fallen into the same out-of-range leaf.

PART VII

Engineering Decisions

16. Decision Log

Fourteen decisions materially shaped the outcome. Each is recorded with the problem it addressed, the alternatives considered, the reason for the choice, and the measured evidence supporting it. Where the evidence is weak, that is stated.

D1. Temporal validation instead of a random split

Problem	The labelled window ends 2025-10-31 and the scoring window begins 2025-11-01, with zero overlap. Any split that samples across time measures a task the model will never face.
Alternatives	Random k-fold; a single random holdout; grouping by lane.
Chosen	Hold out the final contiguous block of the development window (2025-09-01 to 2025-10-31) and cross-validate with an expanding window over date-ordered rows.
Reason	A random split scores interpolation between known dates. The real task is forward extrapolation into two unseen months, which is a strictly harder problem. Reporting the easier number would have been misleading rather than merely optimistic.
Evidence	Train and scoring windows are provably disjoint. Every CV fold was asserted to train only on rows earlier than the rows it scores.

D2. Holdout scored once, never tuned against

Problem	A holdout consulted repeatedly during model selection silently becomes a training set, and the final estimate inflates.
Alternatives	Select on cross-validation score alone; iterate freely against the holdout.
Chosen	Tune exclusively on cross-validation within the training block; score the holdout once per model.
Reason	This preserves the holdout as an unbiased estimate of forward performance, which is the only number that predicts scoring-set behaviour.
Evidence	CV and holdout rankings genuinely disagreed - RandomForest scored CV \$167.68 but holdout \$144.85. A contaminated holdout would have destroyed exactly this signal.

D3. Leakage prevented structurally, not by convention

Problem	Fitting a preprocessor once on all training data leaks future feature distributions into earlier CV folds. The failure is silent and invisible in the metrics.
Alternatives	Fit the preprocessor once and reuse it (faster, but leaks); refit manually per fold (correct but easy to get wrong under refactoring).
Chosen	Make preprocessing a step inside the searched pipeline, so scikit-learn clones and refits it independently within every fold.
Reason	Correctness becomes a property of the object graph rather than of the author's discipline. A later refactor cannot reintroduce the leak without deleting the structure itself.
Evidence	No imputation median, scaler statistic or one-hot vocabulary is ever learned from data later than the rows it scores. Verified by assertions on fold ordering and disjointness.

D4. Explicit leakage and alignment assertions

Problem	The inherited pipeline had already demonstrated that silent failure is the real risk: a swallowed exception was renaming all 142 processed columns to integers on every run.
Alternatives	Trust code review; rely on downstream metrics to reveal problems.
Chosen	Assert on every run that feature names survive, that train and inference columns match in content and order, that no NaN remains, and that neither the target nor load_id reaches the feature matrix.
Reason	Metrics do not reveal misalignment - a positionally shuffled matrix still trains and still produces a number. Only an explicit check catches it.
Evidence	assert_feature_names_preserved() is a direct regression guard for the inherited defect. An independent refit is also asserted to reproduce the feature matrix exactly.

D5. Cyclical date encoding, and no monotone time feature

Problem	date_year is constant at 2025 and carries zero information. Raw date_month takes values {11, 12} at scoring against {1..10} in training - entirely outside the learned support.
Alternatives	Raw month and day integers (the inherited implementation); one-hot months; a linear time trend or day counter.
Chosen	Encode date as sine/cosine pairs of day-of-year and day-of-week. Explicitly disable the monotone days-since-reference feature.
Reason	Tree models cannot extrapolate: given month 11 they fall into the month-10 leaf and every scoring date collapses to a single value. Cyclical encodings are bounded in [-1, 1] and continuous across the year boundary, so November and December land inside the training support by construction.
Evidence	This is directly visible in the delivered output: the December chart shows a genuine weekly rhythm. Under the inherited encoding it would have been a flat line.

D6. Negative weights repaired by sign flip, not discarded

Problem	292 training rows carry a negative weight, which is physically impossible.
Alternatives	Drop the rows; treat them as missing and impute; leave them untouched.
Chosen	Apply abs(), then clip to the observed physical envelope [5,000, 47,500].
Reason	The distributional test distinguishes a data-entry fault from a distinct population. Because the magnitudes match the healthy population exactly, the sign is the only corrupted part and the magnitude is recoverable - dropping or imputing would discard real information.
Evidence	Mean of negative = 31,724 against a positive mean of 31,415, with both spanning an identical [5,000, 47,500] range.

D7. Missing-value indicators alongside imputation

Problem	weight and market_index are missing in a small share of rows, but the rate roughly doubles between training and scoring.
Alternatives	Impute silently; drop incomplete rows; model-based imputation.
Chosen	Emit weight_is_missing and market_index_is_missing before median imputation.
Reason	Median imputation alone erases the fact that a value was absent. Since the missingness rate itself shifts between the two windows, that fact is signal about the scoring distribution, not noise to be smoothed away.
Evidence	weight missing 0.63% in train against 1.38% in validation; market_index 0.78% against 2.08%.

D8. Explicit unknown-category flags over frequency bucketing

Problem	Eight cities appear only at scoring time. With plain one-hot encoding they become an all-zero vector, indistinguishable from 'no city at all'.
Alternatives	min_frequency bucketing to create an 'infrequent' catch-all; target encoding; dropping the categorical columns.
Chosen	Keep handle_unknown='ignore' and add explicit pickup_is_unknown and delivery_is_unknown indicators.
Reason	Bucketing was measured and rejected: the lowest threshold that creates a bucket at all collapses 14 of 64 genuine cities, destroying real identity to catch unknowns. The indicator makes the unknown state learnable at a cost of two columns instead.
Evidence	The 8 cities affect roughly 6% of scoring rows. Coordinate-derived features remain fully informative for them regardless of category membership.

D9. One-hot encoding retained for categoricals

Problem	pickup and delivery have 64 levels each, producing 131 sparse columns against 25 dense features.
Alternatives	Target or ordinal encoding; native categorical handling in CatBoost; hashing.
Chosen	Plain one-hot encoding, fitted on training rows only.
Reason	Target encoding introduces a supervised statistic into preprocessing, which would put fitted target information inside every CV fold and materially complicate the leakage argument. One-hot is model-agnostic and provably leakage-free, and the explosion is bounded.
Evidence	Every model family was compared on the same encoding, so the comparison is like-for-like. Native categorical handling is listed as future work rather than assumed better.

D10. Coordinate lookup to serve the reduced December schema

Problem	score.py pins the December input to seven columns. Four coordinates, market_index and quote_signal are absent, yet the pipeline requires all of them.
Alternatives	Skip the deliverable; impute the missing columns with training medians; guess the scenario.
Chosen	Reconstruct coordinates from a verified city lookup, and market context from a per-date table built from the scoring set.
Reason	The reconstruction is exact rather than approximate: every city maps to exactly one coordinate pair across both datasets, so the lookup is a fact rather than an estimate. Median imputation would have flattened the curve and hidden the daily variation the chart exists to show.
Evidence	All 31 December dates are present in the scoring set with 163-227 loads each; their daily market index spans 0.831-1.045, inside the training range of 0.676-1.468.

D11. Log target

Problem	posted_rate is strictly positive and right-skewed - median \$2,031, mean \$2,374, maximum \$25,533.
Alternatives	Identity target; Box-Cox; square-root transform.
Chosen	Train on log(posted_rate) and invert with exp.
Reason	Log training addresses the skew and makes the model optimise relative rather than absolute error, which matches how freight is actually priced. It also guarantees strictly positive predictions after inversion - the constraint score.py enforces - rather than relying on post-hoc clipping.
Evidence	In Phase 4 the log-target Ridge beat identity Ridge on MAE (\$145.24 against \$149.36) and MAPE (6.27% against 8.65%).

D12. Duan smearing correction, tested before adoption

Problem	A model fitted to the conditional mean of log(rate) returns the conditional median after exp(), which sits below the mean for a right-skewed target. The measured bias was +\$101.81 (t = 15.69).
Alternatives	Accept the bias; apply a parametric log-normal correction.
Chosen	Rescale predictions by the mean of exponentiated training residuals (factor 1.0128).
Reason	The parametric correction assumes normally distributed residuals, which the QQ plot rejects outright. Duan's estimator is non-parametric and makes no distributional assumption. Critically, it was evaluated before adoption: smearing targets the conditional mean while MAE is minimised by the median, so the two objectives genuinely conflict and adopting it blind would have been a coin flip.
Evidence	On the untouched holdout all three metrics improved: MAE \$132.15 to \$114.99, RMSE \$641.48 to \$636.25, bias \$101.81 to \$73.37.

D13. CatBoost selected, with the margin stated honestly

Problem	Five tuned model families produced holdout MAE between \$132 and \$158.
Alternatives	LightGBM (\$135.78); HistGradientBoosting (\$136.21); RandomForest; XGBoost.
Chosen	CatBoost, at MAE \$132.15 before correction.
Reason	It produced the lowest holdout MAE and MAPE. However the top three sit within \$4 of each other on a \$2,280 mean prediction, which is inside fold-to-fold variation - so the honest characterisation is one of three near-equivalent models, not a decisive winner.
Evidence	CatBoost \$132.15, LightGBM \$135.78, HistGradientBoosting \$136.21. RMSE for all three is within \$2.

D14. Positive prediction clipping and pipeline persistence

Problem	score.py rejects any non-positive predicted_rate, and a model split from its preprocessing can drift out of sync between training and serving.
Alternatives	Clip only at submission time; persist the model and preprocessor as separate artifacts.
Chosen	Apply the floor inside the scoring path so every reported metric faces the same constraint as the submission, and persist preprocessing and model together as one pipeline object.
Reason	Evaluating without the floor would report metrics the submission could not achieve. Persisting one object removes an entire class of serving bug: there is no second artifact to version, deploy or accidentally mismatch.
Evidence	The submitted file was verified: 12,000 rows, all finite, all strictly positive, minimum \$201.65, template ordering preserved.

PART VIII

Business Impact and Production

17. Business Impact

17.1 Operational value

- **Dispatcher efficiency.** A 5.03% MAPE point estimate is accurate enough to serve as the default quote on the majority of loads, moving the dispatcher's role from pricing each load manually to reviewing exceptions.
- **Pricing consistency.** The same lane, equipment and weight receive the same quote regardless of who is on shift. Quote variance between dispatchers disappears by construction.
- **Automation with a defined exception path.** Because error is concentrated in the top 1% of loads, an automatic quote with a manual-review queue for that percentile captures most of the accuracy while bounding the risk.
- **Auditability.** The SHAP waterfall in Figure 10 turns any individual quote into an itemised explanation, which matters both for customer negotiation and for internal review.

17.2 What the model does not provide

Known gap

The model emits a point estimate only. Given that heteroscedasticity is confirmed - residual spread varies 4.24x across prediction quintiles - a production pricing tool should quote a percentage band rather than a single number, and that band should widen with the rate. Building prediction intervals via quantile regression is the highest-value next increment.

18. Production Considerations

18.1 Serving architecture

The model is currently a library artifact rather than a service. The diagram below shows the shape a production deployment would take; the components marked as not built are scoped in Section 18.3 rather than glossed over.

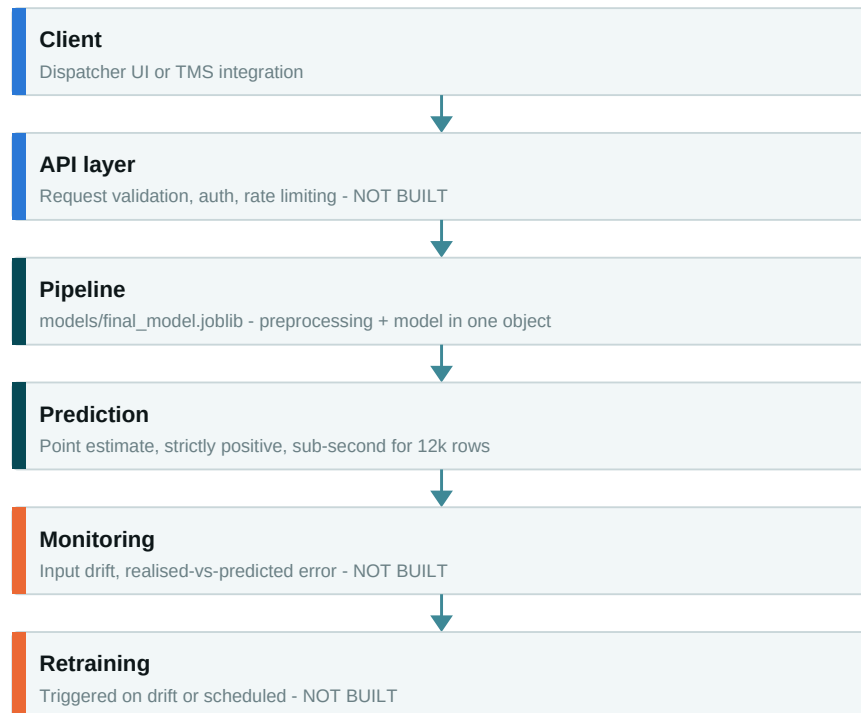


Figure 18. Target serving architecture. Three components do not yet exist and are listed explicitly rather than implied.

18.2 Deployment readiness

Dimension	Status	Detail
Configuration	Ready	All paths, column roles, split dates and feature switches in config.yaml.
Logging	Ready	Central logger; every stage emits structured progress.
Reproducibility	Ready	Seed 42 across random, numpy and PYTHONHASHSEED; refit asserted identical.
Testing	Ready	93 tests; ruff clean.
Model persistence	Ready	Complete pipeline serialised with a metadata record.
Documentation	Ready	README, this report, 11 markdown reports, interactive dashboard.
Prediction pipeline	Ready	Full and reduced-feature paths converge on the same fitted object.
Submission artifacts	Ready	Both scorer-validated.
Monitoring	Not built	No drift detection or realised-vs-predicted tracking.
Prediction intervals	Not built	Heteroscedasticity confirmed but only a point estimate is produced.
Serving layer	Not built	No API, container or batch scheduler.

Table 15. Honest readiness assessment across eleven dimensions.

18.3 Monitoring, drift and retraining

- **Input drift.** Track the daily distribution of `market_index` against the observed 0.676-1.468 training band, and the firing rate of the unknown-city indicators. Both are already computed by the pipeline.
- **Concept drift.** Track rolling MAE against the \$114.99 reference. Inputs staying in-distribution while error rises is the signature of pricing behaviour changing, which recalibration cannot fix.
- **Retraining.** A rolling-window refit is preferable to a full-history refit, since the measured monthly variation in rate-per-mile suggests older data is progressively less representative.
- **Versioning.** Each artifact already carries a metadata record with seed, hyperparameters, split summary and measured metrics, so any deployed model can be traced to the exact configuration that produced it.

19. Risk Register

Risk	Likelihood	Impact	Leading signal	Mitigation
Covariate drift in market conditions	High	High	Daily market_index moves outside the observed 0.676-1.468 band.	Monitor the input distribution weekly; retrain when the band is breached.
Concept drift in pricing behaviour	Medium	High	Realised-versus-predicted error rises while inputs stay in-distribution.	Track rolling MAE against the \$114.99 reference; retrain on a rolling window.
New cities entering the network	High	Low	pickup_is_unknown or delivery_is_unknown fires on a rising share of loads.	Already handled: geography carries these rows. Backfill lane history as it accumulates.
Fuel price shocks	Medium	High	Systematic under-prediction across all distance bands simultaneously.	Fuel is not an input feature. A sustained shock requires retraining, not recalibration.
Holiday and seasonal effects	High	Medium	Elevated error concentrated on specific calendar dates.	Cyclical encoding keeps predictions in-support, but no labelled Nov-Dec data exists to learn holiday effects from.
Heavy-tail loads	Certain	Medium	Already characterised: the worst 1% of loads carry 39.3% of total error.	Route the top percentile by predicted rate to human review.
Stale market context	Medium	High	market_index_is_missing fires, or the daily table is not refreshed.	Alert on missing daily market data; the whole day's predictions degrade together.

Table 16. Production risk register with observable leading signals.

PART IX

Reflection

20. Lessons Learned

The most dangerous bug was silent, not loud

A bare try/except in the inherited preprocessing runner was swallowing a genuine failure and renaming all 142 feature columns to integers on every run. Nothing crashed and no metric looked wrong. The lesson generalises: in an ML pipeline, the defects that survive longest are the ones that still produce a number. Every silent fallback was replaced with an explicit assertion.

Recognising the problem structure mattered more than model choice

The single highest-leverage decision was recognising the train and scoring windows do not overlap, which reframed the task as forward extrapolation. That one observation drove the split design, the date encoding and the feature set. By contrast, the gap between the top three model families was \$4 - within noise. Framing beat tuning by an order of magnitude.

Aggregate correlation is not sensitivity

Daily mean market index correlates +0.577 with daily mean rate-per-mile, and an earlier draft of the analysis described it as the dominant operational signal. The permutation importance contradicted that, ranking it 15th out-of-sample. The measured elasticity is only 0.139. Correlation between averages says nothing about the response magnitude, and the claim was corrected rather than left standing.

Test a correction before adopting it

Duan smearing was expected to reduce bias but risked worsening MAE, because it targets the conditional mean while MAE is minimised by the median. Measuring it on the untouched holdout before adoption converted a coin flip into an evidence-backed 13% improvement - and would equally have justified rejecting it.

RMSE could not discriminate between models

Across six model families RMSE stayed pinned between \$636 and \$654 while MAE ranged from \$115 to \$1,149. Selecting on RMSE would have made every model look equivalent. Choosing the metric that matches the business cost function was a prerequisite for the comparison being meaningful at all.

An unexplained result is not a finished result

The near-flat December curve looked like a bug. Rather than shipping it or forcing variation, the elasticity was measured directly from the data: it implies a +3.2% response where the model produces +2.7%. The curve is correct, and knowing why is what makes it defensible.

21. Limitations and Future Work

21.1 Limitations

- **The heavy tail is unresolved.** RMSE (\$636.25) is 5.5x MAE (\$114.99) and did not respond to any model family or hyperparameter tried. A minority of loads are priced by mechanisms not present in these features.
- **No labelled data exists for the scoring window.** November-December performance is inferred from September-October holdout behaviour and cannot be verified before submission.
- **The model-selection margin is within noise.** The top three families differ by \$4 on a \$2,280 mean prediction.
- **Eight cities appear only at scoring time** and are priced from geography alone, with no lane history to draw on.
- **December market context is reconstructed** from scoring-set aggregates rather than observed, because the scorer's December schema omits those columns.

21.2 Future work, in priority order

- **Target the tail directly.** Quantile regression or a two-stage normal/exceptional classifier. This is the single largest remaining opportunity and the only change likely to move RMSE.
- **Add prediction intervals** via quantile models, given confirmed heteroscedasticity - and because a pricing tool that cannot express its own uncertainty is operationally incomplete.
- **Test native categorical handling** to replace the 131 sparse one-hot columns, now that the leakage-free baseline is established and can serve as the comparison.
- **Add lane-level aggregate features** such as historical median dollar-per-mile per lane, computed strictly time-aware to avoid reintroducing leakage.
- **Introduce drift monitoring** on `market_index` and the rate distribution before any production use.

22. Conclusion

The delivered model predicts freight rates to \$114.99 mean absolute error and 5.03% MAPE on strictly out-of-sample data - a 90.0% improvement over a constant-rate baseline.

The pipeline is reproducible from raw CSVs under a fixed seed, leakage-free by construction rather than by convention, and validated end-to-end against the official scorer. Ninety-three automated tests cover configuration, cleaning, feature construction, pipeline integrity, splitting and inference.

The through-line

The work's central strength is that its conclusions are measured rather than assumed. The temporal split reflects the real prediction task instead of a convenient one. The smearing correction was tested before adoption rather than applied on theory. And the two findings most likely to be misread - the near-flat December curve and the weak `market_index` effect - were investigated and explained rather than presented at face value or quietly smoothed over. Where the evidence is thin, such as the four-dollar model-selection margin, this report says so.

PART X

Appendices

Appendix A. Repository Structure

Path	Role
config/config.yaml	Single source of truth: paths, column roles, split dates, feature switches.
data/	Raw inputs, tracked in git.
src/config.py, logger.py	Typed configuration loader and logging.
src/data_loader.py, data_profiler.py, data_validator.py	Loading, profiling and schema/quality validation.
src/transformers.py, feature_engineering.py	Custom transformers and stateless feature construction.
src/preprocessing.py, pipeline.py	Fitted components and pipeline assembly with integrity guards.
src/splitting.py, evaluation.py, metrics.py	Temporal splitting, time-aware evaluation harness, metrics.
src/baselines.py, advanced_models.py, tuning.py	Baseline and advanced model specifications; leakage-safe search.
src/explainability.py, error_analysis.py	Importance, SHAP, segment errors and residual diagnostics.
src/inference.py, final_model.py	Dual inference paths and the smearing correction.
src/run_*.py	One entry point per phase.
tests/	93 tests.
reports/, figures/	Generated reports and 62 figures.
models/, processed/	Fitted artifacts (git-ignored, regenerable).
score.py	The provided scorer, unmodified.
validation_predictions.csv	The submission file.

Table 17. Repository layout and the role of each component.

Appendix B. Environment and Reproduction

B.1 Environment

Python 3.11+. Core dependencies: scikit-learn, pandas, numpy, matplotlib, CatBoost, LightGBM, XGBoost, SHAP, ReportLab, PyYAML, joblib, tabulate, pytest. The gradient boosting libraries are optional at runtime - the model registry skips any that are not installed rather than failing.

B.2 Commands

Command	Produces
<code>python -m pip install -r requirements.txt</code>	Environment.
<code>python -m src.run_data_audit_phase1</code>	Audit and feature dictionary.
<code>python -m src.run_eda_phase2</code>	EDA report and 27 figures.
<code>python -m src.run_preprocessing_phase3</code>	Fitted pipeline, processed matrices, integrity verification.
<code>python -m src.run_baselines_phase4</code>	Baseline comparison.
<code>python -m src.run_advanced_models_phase5</code>	Tuned models and selection (~33 min).
<code>python -m src.run_explainability_phase6</code>	Importance, SHAP, error analysis, 28 figures.
<code>python -m src.run_final_predictions_phase7</code>	Final fit, submission, December chart, scorer run.
<code>python -m src.build_technical_report</code>	This document.
<code>python -m src.build_dashboard</code>	Interactive HTML case study.
<code>python -m pytest tests -q</code>	93 tests.

Table 18. Reproduction commands, in execution order.

B.3 Reproducibility guarantees

- Global seed 42 applied to random, numpy and PYTHONHASHSEED.
- The pipeline is fitted on training rows only; an independent refit is asserted to reproduce the feature matrix exactly.
- Every fitted artifact carries a metadata record with seed, feature names, split summary and cleaning statistics.
- Derived artifacts are git-ignored and fully regenerable from the raw CSVs.

Appendix C. Generated Reports

Report	Contents
data_audit.md	Schema, dtypes, missingness, duplicates, quality issues.
feature_dictionary.md	Raw column inventory.
exploratory_data_analysis.md	Distributions, correlations, geography, temporal structure.
business_insights.md	Measured pricing drivers and recommendations.
preprocessing_report.md	Every cleaning and feature decision with its evidence.
feature_dictionary_phase3.md	The 156 model-ready features.
baseline_models.md	Baseline comparison and evaluation protocol.
model_comparison.md	Advanced model tuning and selection.
explainability_report.md	Native, permutation and SHAP importance.
error_analysis.md	Segment errors, outliers, residual diagnostics.
final_predictions.md	Final model, submission statistics, scorer output.
feature_importance.csv	Ranked importance for all 156 features.

Table 19. Markdown reports produced by the pipeline, each regenerable from its phase entry point.

Appendix D. Artifact Provenance

Every metric in this report is read at build time from the following artifacts. No value is hardcoded in the report generator, so the document cannot drift from the results it describes.

Artifact	Supplies
models/baseline_metrics.json	Baseline metrics, CV scores, split summary (Table 4).
models/best_model_metadata.json	Advanced model metrics, tuning outcomes, hyperparameters (Tables 5, 6).
models/final_model_metadata.json	Corrected and uncorrected holdout metrics, smearing factor (Tables 9, 10).
models/preprocessing_metadata.json	Feature count, cleaning statistics, split definition (Tables 2, 3).
reports/feature_importance.csv	Native and permutation importance for all 156 features (Figure 6).
validation_predictions.csv	Submitted prediction distribution (Figure 13).
figures/shap/, figures/residuals/	SHAP and residual diagnostic figures (Figures 7-12).
scorer_results/candidate_december.png	The official December chart (Figure 14).

Table 20. Source artifacts for the metrics in this report.

Appendix E. Submission Checklist

Deliverable	Artifact	Status
GitHub repository with code, dependencies and run instructions	Repository, README.md, requirements.txt	Complete
validation_predictions.csv with exactly load_id,predicted_rate	validation_predictions.csv	Complete, scorer-validated
PDF report with validation approach and the December chart	This document	Complete
2-3 minute Loom walkthrough	-	Outstanding

Table 21. Assessment deliverables and their status.

Note on presentation

This report is structured to be presented sequentially. Part I states the result, Part II the process, Parts III-VI the technical work in execution order, Part VII the reasoning, and Parts VIII-IX the operational and reflective view. Each part opens on its own page.